



ADRIANO MOURA DA SILVA

**RELATÓRIO – DESENVOLVIMENTO FRONT-END E CONSUMO DE API
ASSÍNCRONA**

Taboão da Serra, SP

2026

O que é uma API e o Consumo de Dados em Tempo Real

No desenvolvimento web moderno, as aplicações raramente funcionam de forma isolada. Para criar uma experiência rica e atualizada para o usuário, os sites precisam se conectar a serviços externos que gerenciam e fornecem dados. É exatamente aí que entram as APIs (do inglês, *Application Programming Interface*, ou Interface de Programação de Aplicações).

Uma API funciona como uma ponte ou um "garçom" entre o meu código de Front-End e um servidor de Banco de Dados. Em vez de eu salvar centenas de informações e imagens diretamente nos meus arquivos locais, o meu código faz uma requisição (um pedido) para um endereço web específico da API, e ela me devolve esses dados estruturados, geralmente no formato JSON (JavaScript Object Notation).

O grande diferencial do consumo de dados em tempo real e de forma assíncrona é que a página não precisa ser completamente recarregada para mostrar novas informações. Quando o usuário digita na barra de pesquisa ou altera um filtro, o JavaScript intercepta essa ação, faz uma nova requisição em segundo plano, processa a resposta e atualiza apenas a parte necessária da tela. Isso torna a navegação extremamente rápida, fluida e dinâmica.

Manipulação do DOM (Document Object Model) e Elementos Dinâmicos

Para que os dados recebidos da API apareçam na tela, o JavaScript precisa interagir com a estrutura da página web. Essa estrutura é representada pelo DOM (Document Object Model), que é, basicamente, a árvore de elementos HTML que o navegador cria ao ler o código do site.

A manipulação do DOM acontece quando usamos o JavaScript para alterar, adicionar ou remover partes dessa árvore em tempo real. No desenvolvimento deste projeto, adotei uma abordagem focada em boas práticas de desempenho e segurança. Em vez de utilizar a propriedade `innerHTML` de forma massiva, que força o navegador a destruir e reconstruir toda a estrutura interna de texto como HTML, deixando a página pesada e

vulnerável, optei por gerar os cards dos personagens de maneira 100% dinâmica através de objetos nativos do ecossistema do DOM.

Ao criar elementos usando métodos estruturados, o código cria nós diretamente na memória do navegador antes de jogá-los na tela. Isso garante que a interface responda de forma muito mais rápida, mesmo lidando com um volume alto de dados.

Funções Nativas Utilizadas no Desenvolvimento

Para dar vida à lógica da aplicação, utilizei um conjunto de funções essenciais do JavaScript nativo (Vanilla JS), cumprindo os requisitos de não utilizar frameworks externos:

- *fetch()*: É o método assíncrono padrão do JavaScript usado para disparar as requisições HTTP para a API. Combinado com a sintaxe moderna de *async/await*, ele me permite "esperar" a resposta do servidor sem travar a execução do resto do site.
- *Promise.all()*: Como a PokéAPI fornece uma lista inicial contendo apenas os nomes e as URLs de cada criatura, precisei disparar requisições internas para buscar os detalhes individuais (como fotos e status). O *Promise.all* foi fundamental para agrupar todas essas requisições paralelas e esperar que todas terminassem juntas, otimizando o tempo de carregamento.
- *document.createElement()*: Função utilizada para criar novas tags HTML (como *div*, *img*, *h5* e *span*) diretamente pelo JavaScript.
- *appendChild()* e *append()*: Métodos utilizados para estruturar a hierarquia dos cards. O *append()* me permitiu encaixar o cabeçalho, a imagem e o rodapé dentro do card principal de uma só vez, e o *appendChild()* inseriu o card finalizado dentro do container principal da página.
- *replaceChildren()*: Uma técnica de alta performance utilizada para limpar o container de Pokémon sempre que o usuário faz uma nova busca ou

muda de página. Ela remove todos os elementos filhos diretamente da memória de forma muito mais rápida que o tradicional *innerHTML* = "".

Escolha da API e Regras de Acesso

Para o escopo deste desafio, selecionei a PokéAPI (<https://pokeapi.co/>). O motivo da escolha se deu por ser uma API pública de altíssima fidelidade, rica em dados estruturados e com uma identidade visual que permitiu explorar cores e elementos gráficos muito marcantes no CSS, mantendo o apelo nostálgico dos anos 90 e 2000 exigido pelo projeto.

Regras de Acesso e Documentação:

- **Autenticação:** A PokéAPI é totalmente aberta e gratuita, o que significa que ela não exige chaves privadas ou tokens de acesso (*api_key*) para realizar requisições de leitura, facilitando a integração direta no Front-End.
- **Limitação de Requisições:** Embora não haja uma trava rígida de token, a boa prática da documentação recomenda evitar abusos de requisições repetitivas. Para respeitar isso e garantir a boa usabilidade, limitei a busca inicial do meu projeto a 200 registros (*?limit=200*) e criei uma lógica de paginação customizada.
- **Estrutura de Resposta:** O retorno da API entrega objetos em formato JSON. Cada Pokémon possui um nó de tipos (*types*), status base (*stats* para HP e velocidade), e um nó de imagens (*sprites*), onde capturei a propriedade *official-artwork* para exibir as ilustrações em alta definição nos cards, além de buscar as descrições em português na rota auxiliar de espécies.